PARALLEL & DISTRIBUTED COMPUTING · CLASS PROJECT

Biliran Resort & Tour Booking System

Conceptual Framework and Flowcharts · Design v2 · 29 October 2026

1. Conceptual framework

The study follows the Input–Process–Output (IPO) model. Inputs from the four user groups are processed by the distributed nodes using parallel and concurrency-safe techniques, producing consistent bookings, confirmations and reports. Evaluation results feed back into the design.

INPUT

- **Customers:** date, number of guests, chosen package, guest name, e-mail, phone, ID type and last 4 digits, (simulated) payment
- **Receptionists:** walk-in guest details, arrival look-ups, unavailable services and dates
- **Node admins:** services, capacities, packages, prices, receptionist accounts
- **Central admin:** island registrations, node admin accounts, report requests
- **Network state:** island online / offline

→

PROCESS

- **Parallel search fan-out:** central queries all island nodes concurrently via gRPC with deadlines
- **Atomic hold / confirm:** island reserves every service of a package in one transaction (guarded updates)
- **Saga:** hold → pay → confirm, with release / expiry as compensation
- **Idempotent requests:** retries never create duplicates
- **Single-writer ownership** of each data item
- **Outbox synchronisation** island → central; directory pull central → island
- **Offline operation** of island nodes (partition tolerance)
- **Parallel report generation** with worker threads

→

OUTPUT

- Confirmed bookings with confirmation information (reference, package, dates, amount)
- Inventory that is never oversold
- Guests checked in exactly once
- Up-to-date catalog and availability
- Province-wide booking, occupancy and sales reports
- Measured results: speedup, throughput, latency, oversell count (0)

Feedback / evaluation: load tests, race tests, partition (offline) tests, speedup and efficiency measurements, and user feedback from receptionists and admins are used to refine the design.

1.1 Key concepts applied

Concept	How it appears in the system
Distribution / data partitioning	Data partitioned by island; each node stores and owns its own island's data.
Remote procedure call	gRPC contract (`proto/booking.proto`) between central and island nodes.
Parallelism (task)	Concurrent RPCs to all islands during search; total time ≈ slowest island instead of the sum.
Parallelism (data)	Report data split across worker threads (1 / 2 / 4 / 8) and merged.
Concurrency control	Atomic conditional updates and transactions; single writer per data item.
Fault tolerance / partition	Deadlines, offline mode, outbox with retries, de-duplication by event id.
Consistency trade-off (CAP)	Consistency chosen over availability for online sales of an unreachable island.

2. Flowcharts

2.1 Online booking (customer → central → island)

Part 1 — Parallel search

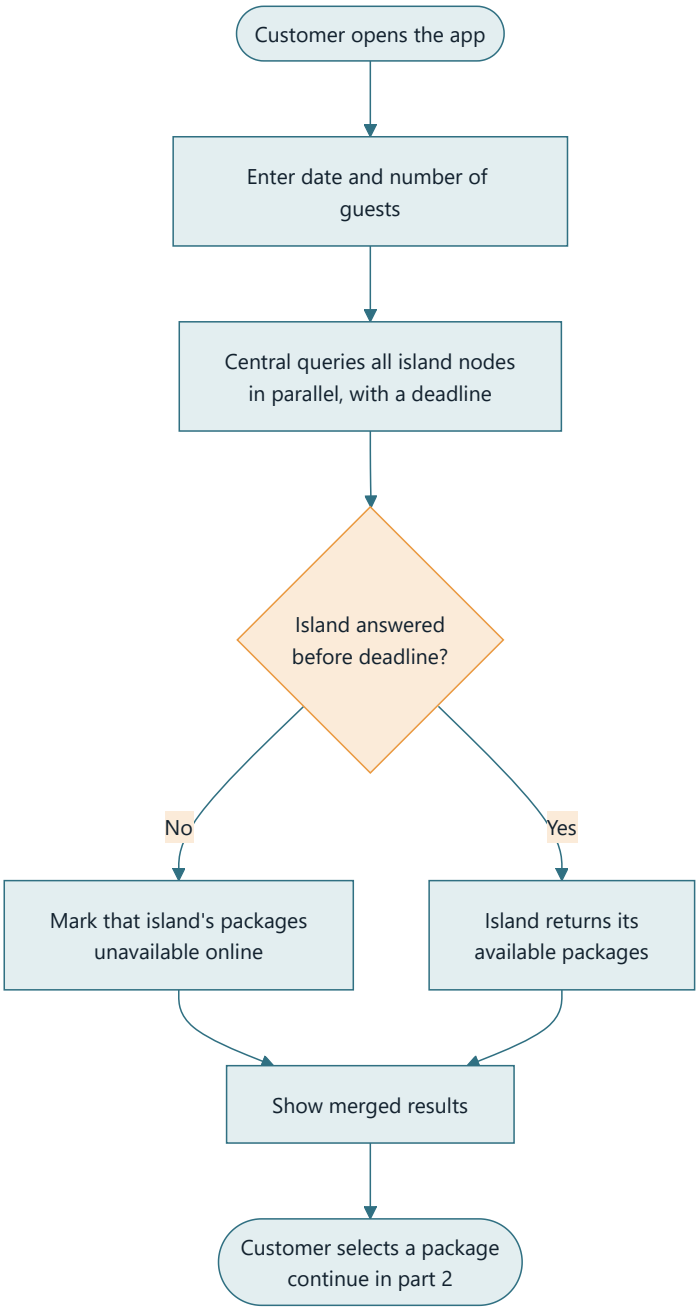


Figure 1a. Central asks every island at the same time; an island that misses the deadline is shown as unavailable

Part 2 — Hold, pay, confirm (saga)

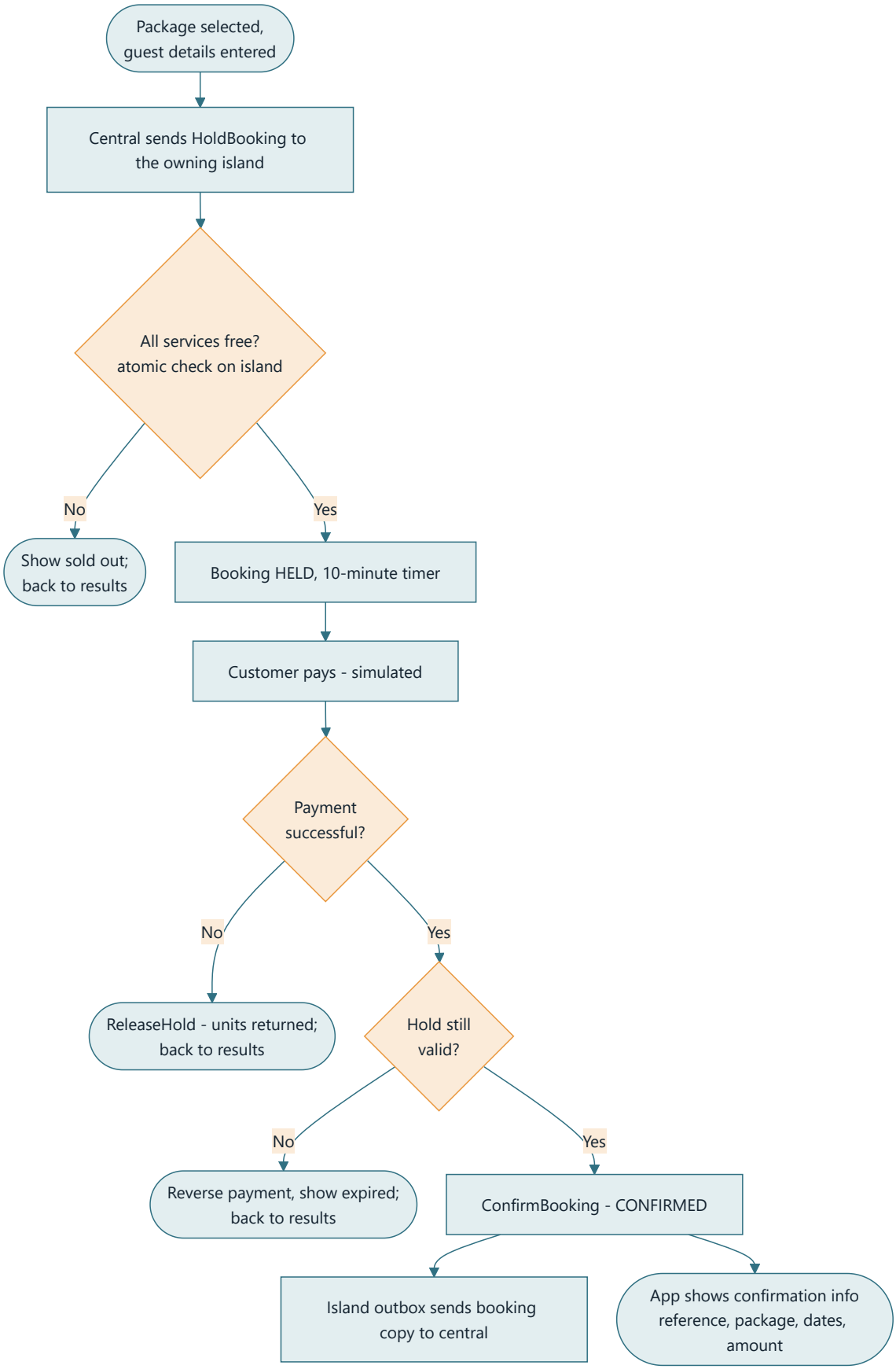


Figure 1b. The owning island reserves all services atomically; release or expiry undoes the hold

2.2 Walk-in booking (receptionist)

> Online and walk-in bookings both go through the island node that owns the inventory, so the race for the last unit is decided by one atomic update on one database.

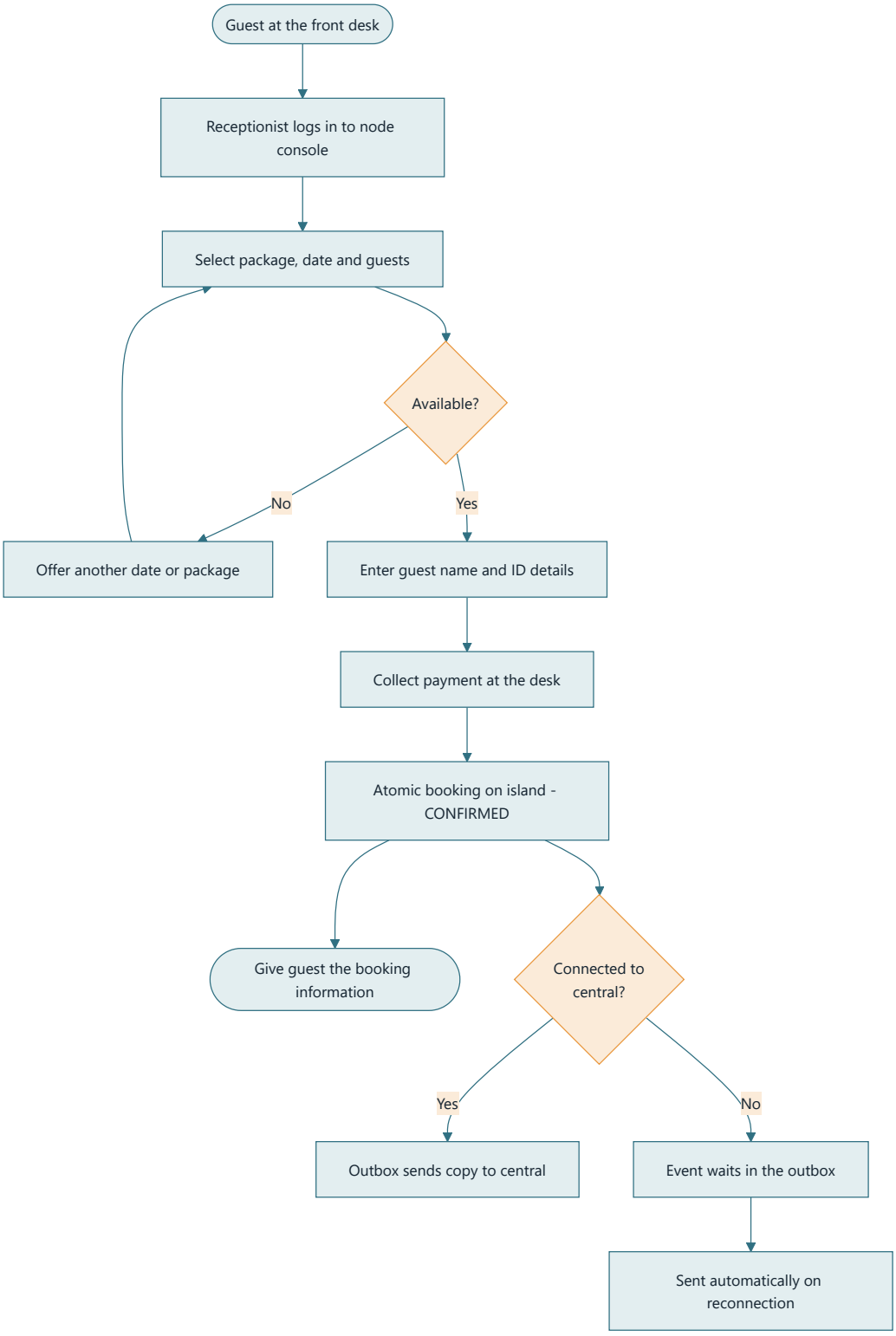


Figure 2. Walk-in booking from the same inventory as online booking

2.3 Arrival confirmation (no tickets)

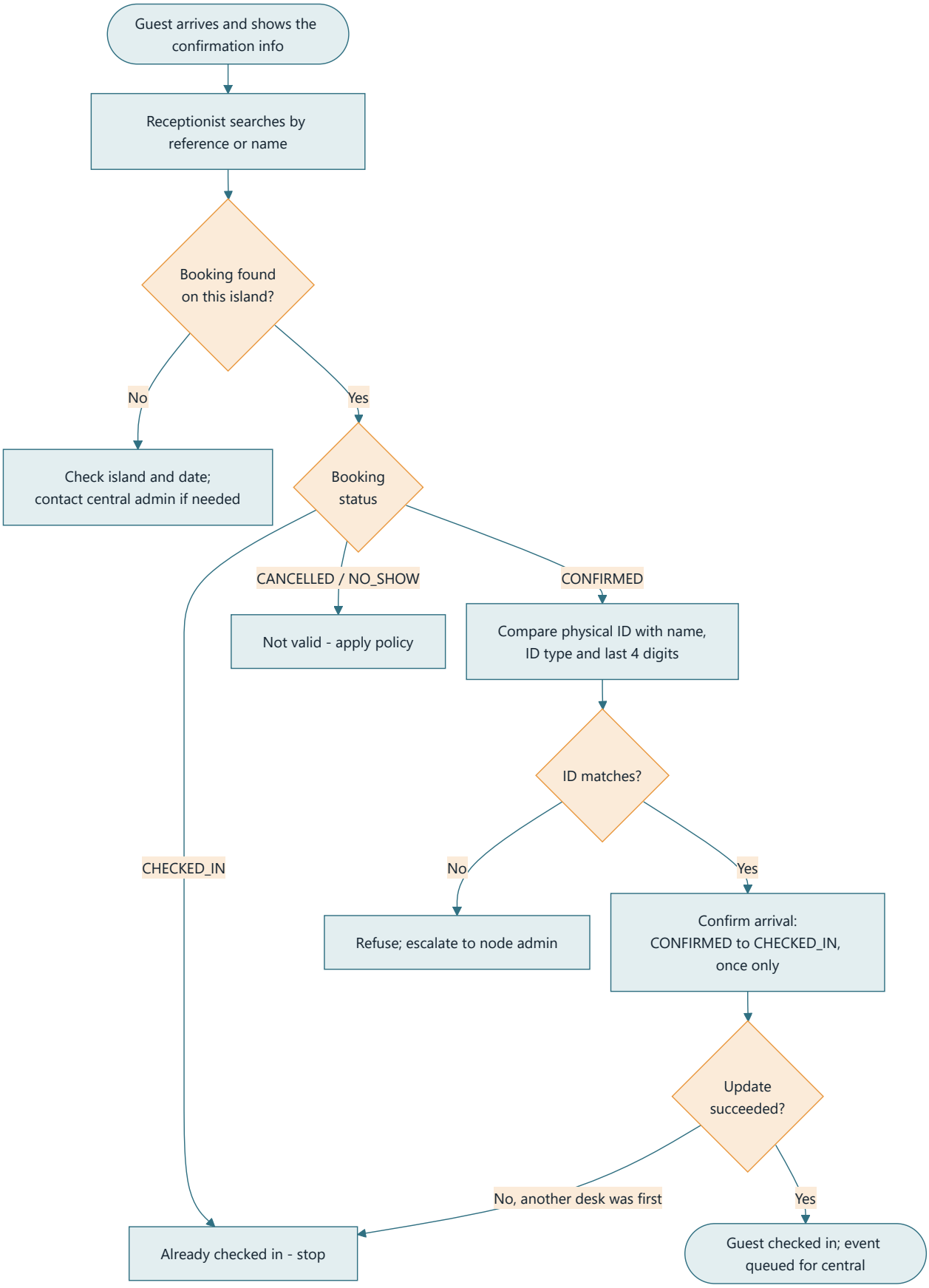


Figure 3. The guest shows the confirmation information received after payment; staff verify the ID

2.4 Mark service unavailable (receptionist)

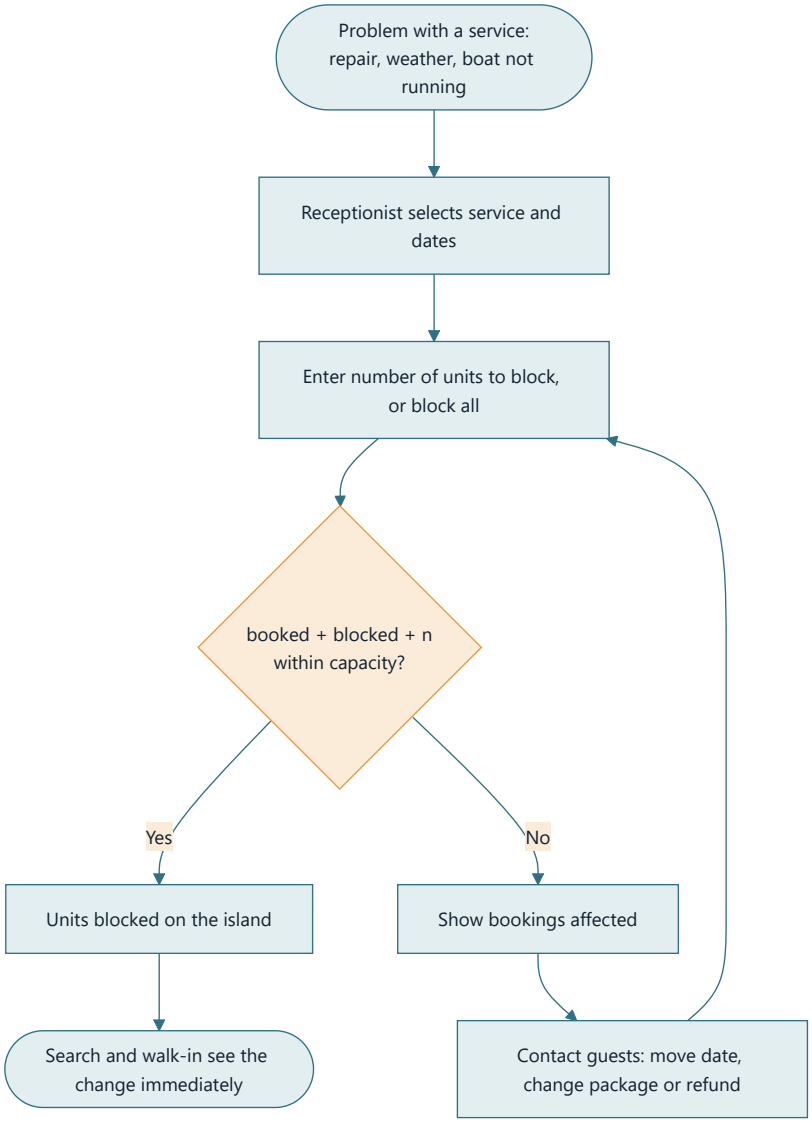


Figure 4. Manual input of unavailable services (repairs, weather, boat not running)

2.5 Catalog change by node admin

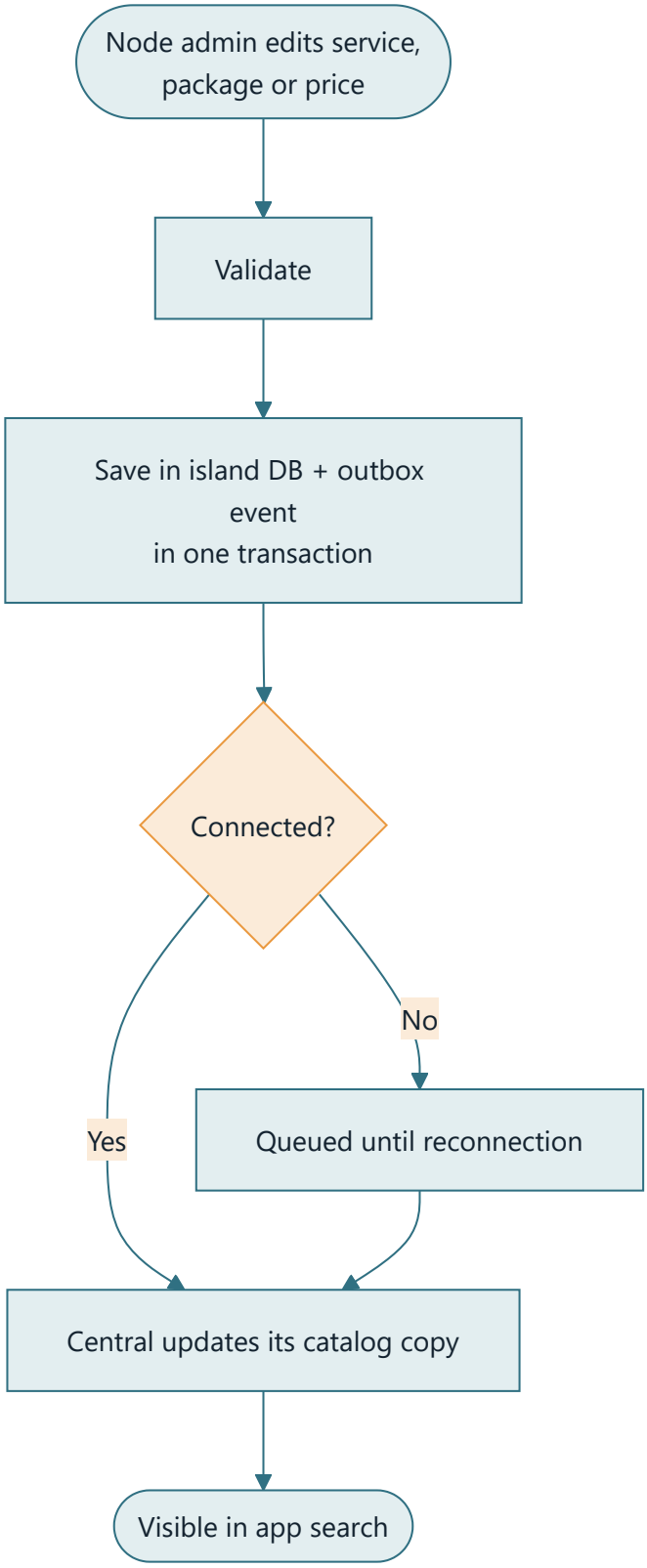


Figure 5. Changes are saved locally first (works offline), then synchronised to central

2.6 Island registration and accounts

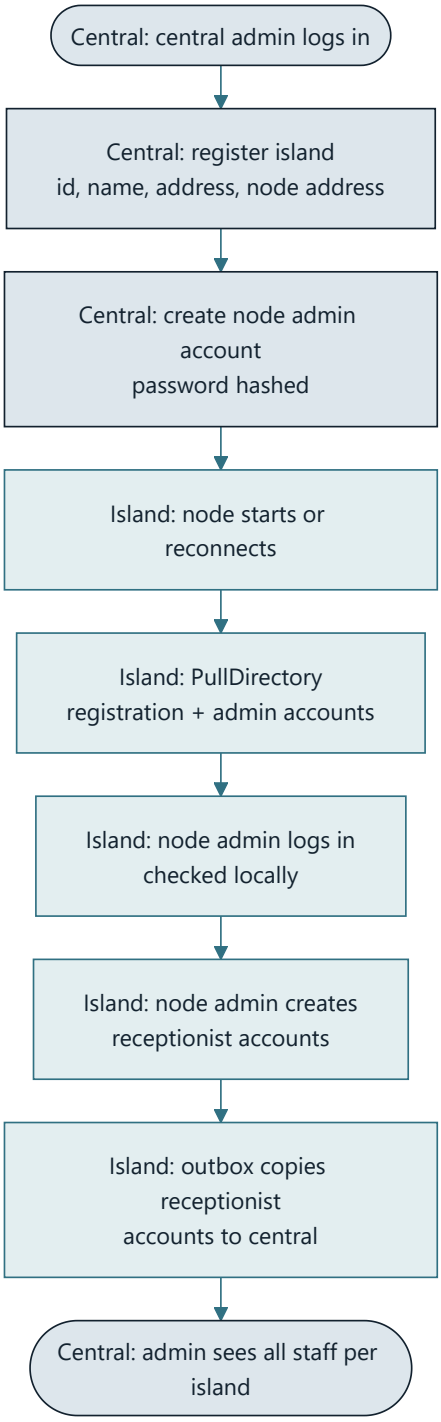


Figure 6. Central admin registers islands and node admins; node admins create receptionists (dark = central, teal = island)

Login

Passwords are stored only as salted scrypt hashes. Each node verifies logins against its local copy of the accounts, so staff can log in while the island is offline. Sessions expire after inactivity; disabled accounts stop working at the next directory pull.